

Deep Generative Models

Dr. Kavita Mohite (Bhosle)

Introduction to deep generative model,
Boltzmann Machine,
Deep Belief Networks,
Generative adversarial network (GAN),
Discriminator network,
Generator network,
Types of GAN,
Applications of GAN networks

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

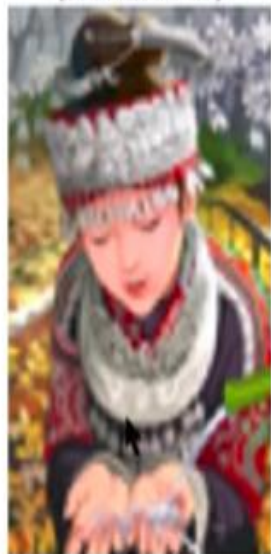
Data: x

Just data, no labels!

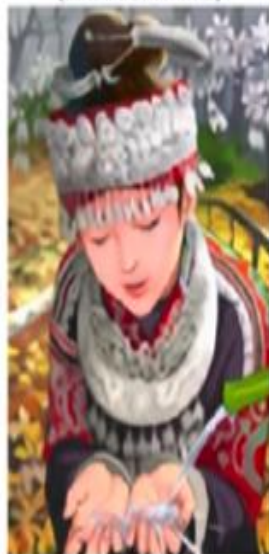
Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

bicubic
(21.59dB/0.6423)



SRResNet
(23.53dB/0.7832)



SRGAN
(21.15dB/0.6868)

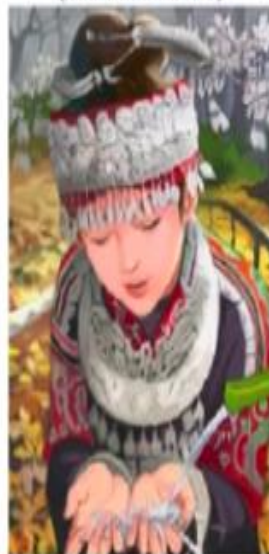
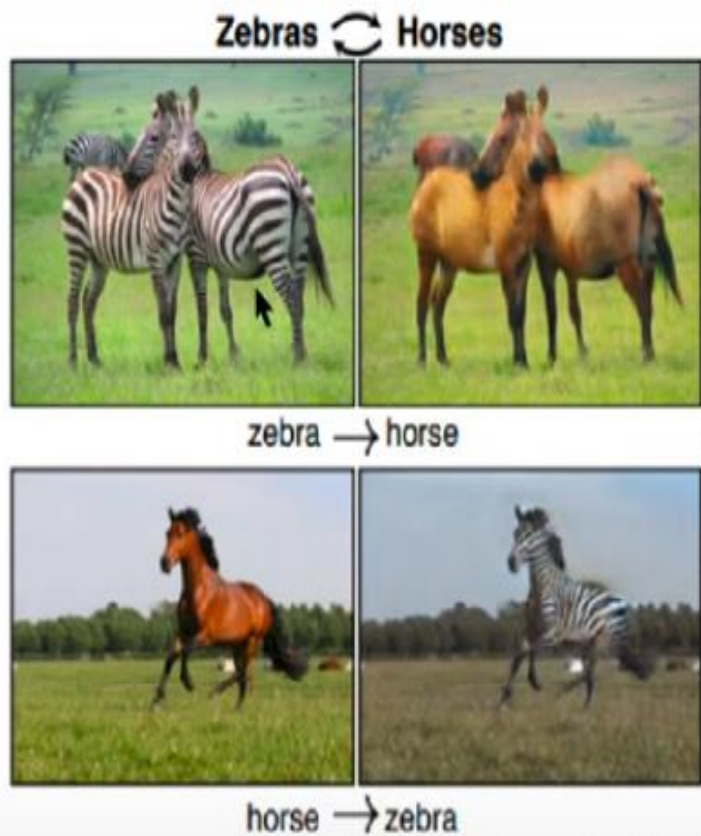


Image Super Resolution



Image Colorization



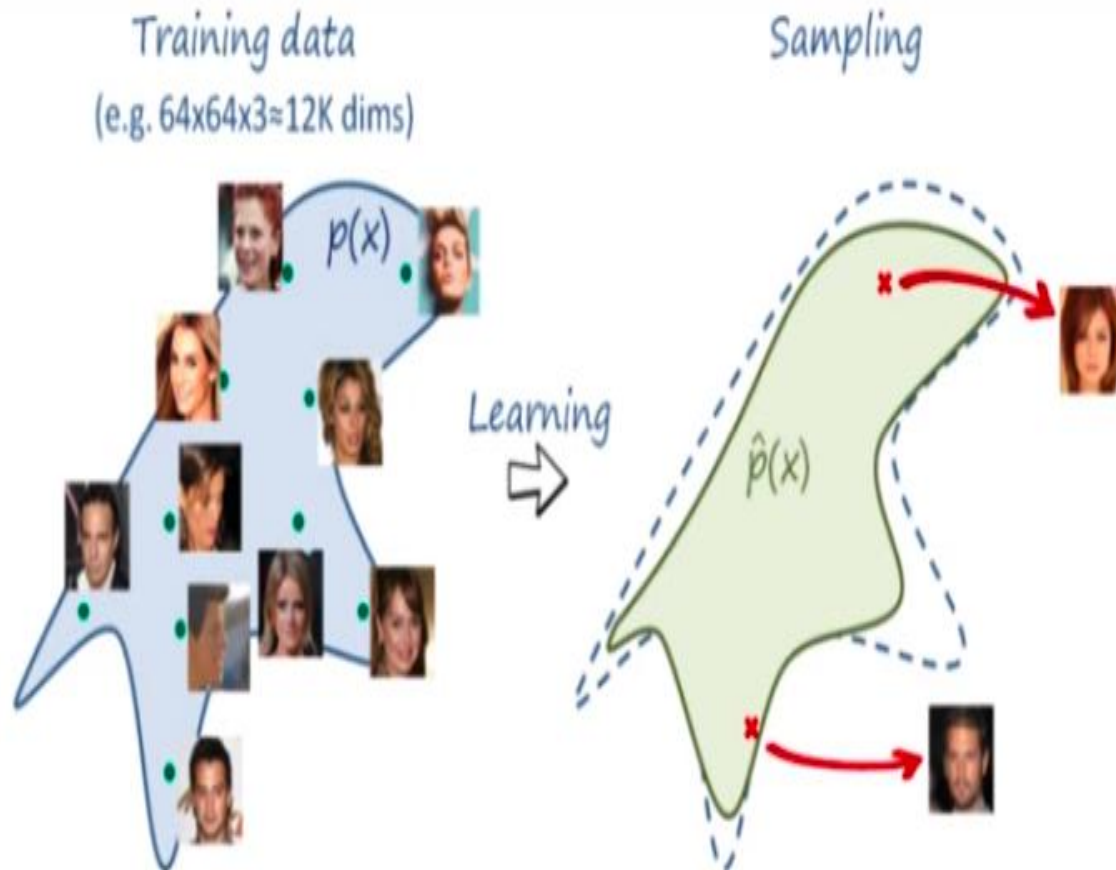
Cross-domain Image Translation



Generating Realistic Face Datasets

- Learn good generalizable latent features
- Augment small datasets
- Enable mixed-reality applications such as Virtual Try-on
- Many more...

- **Goal:** Generate data samples similar to the ones in the training set



- Assume training data $X = \{x_1, x_2, \dots, x_n\}$ comes from an underlying distribution $p_D(x)$, and a generative model samples data from a distribution $p_M(x)$
- Our aim is to **minimize** some notion of **distance** between $p_D(x)$ and $p_M(x)$

Discriminative Model:

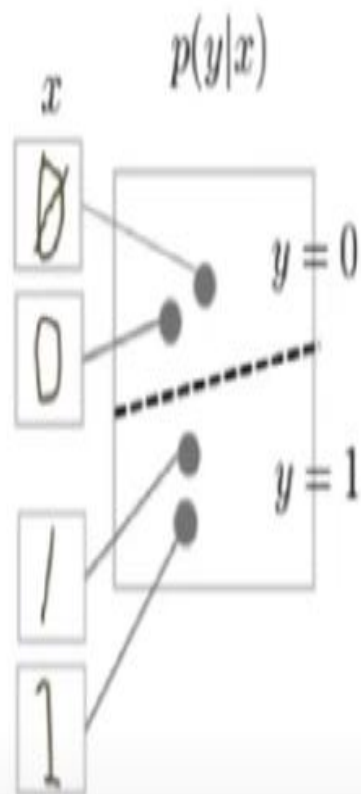
Learn a probability
distribution $p(y|x)$

Generative Model:

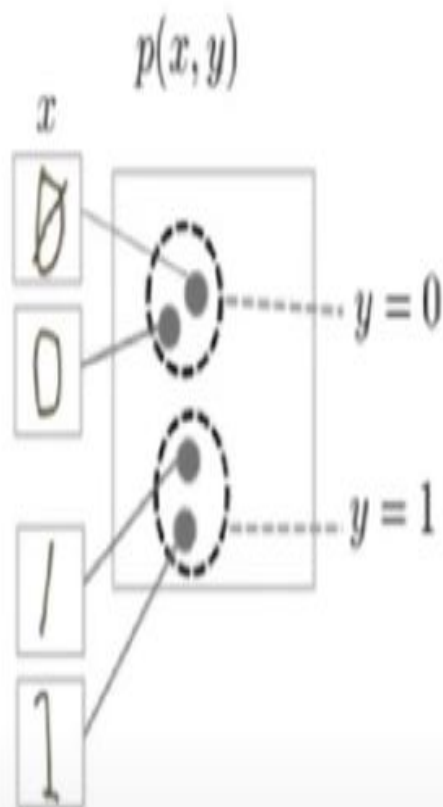
Learn a probability
distribution $p(x)$

**Conditional Generative
Model:** Learn $p(x|y)$

- Discriminative Model

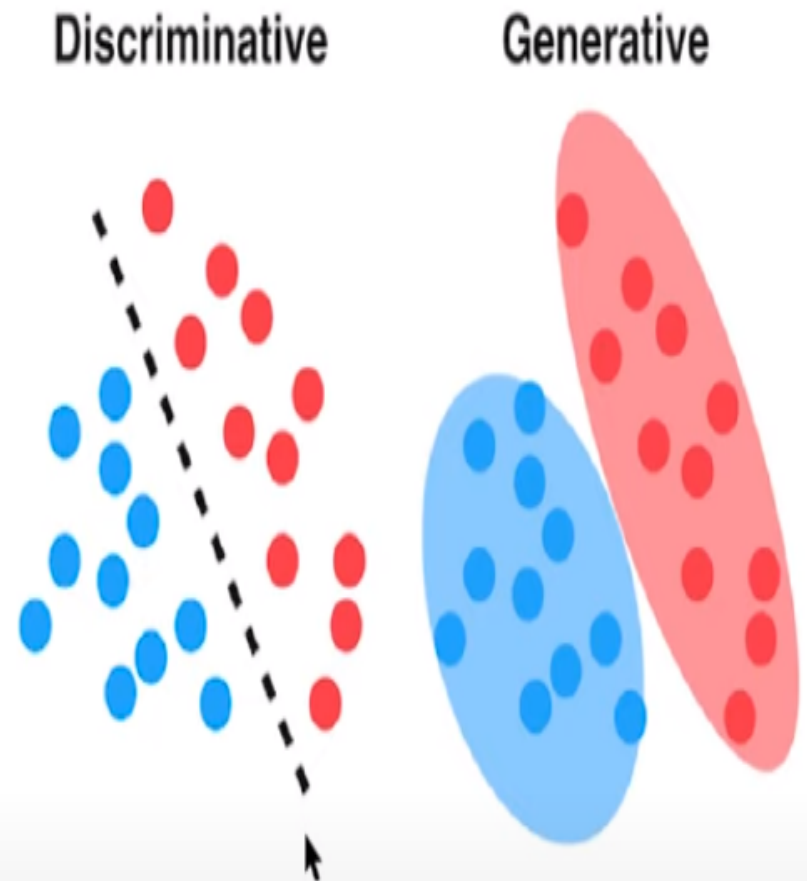


- Generative Model



- Consider a binary classification problem of classifying images of 1s and 0s
- A **discriminative classifier** directly models the **posterior** i.e. $p(y|x)$; x is always given as input
- A **generative classifier** models the **joint distribution** i.e. $p(x, y)$

- **Discriminative** models: aim to learn **differentiating features** between various classes in a dataset
- **Generative** models aim to learn **underlying distribution** of each class in a dataset





$P(\text{cat} | \text{image})$



$P(\text{dog} | \text{image})$

$P(\text{cat} | \text{image})$



Discriminative model: the possible labels for each input "compete" for probability mass.
But no competition between **images**

Discriminative Model:

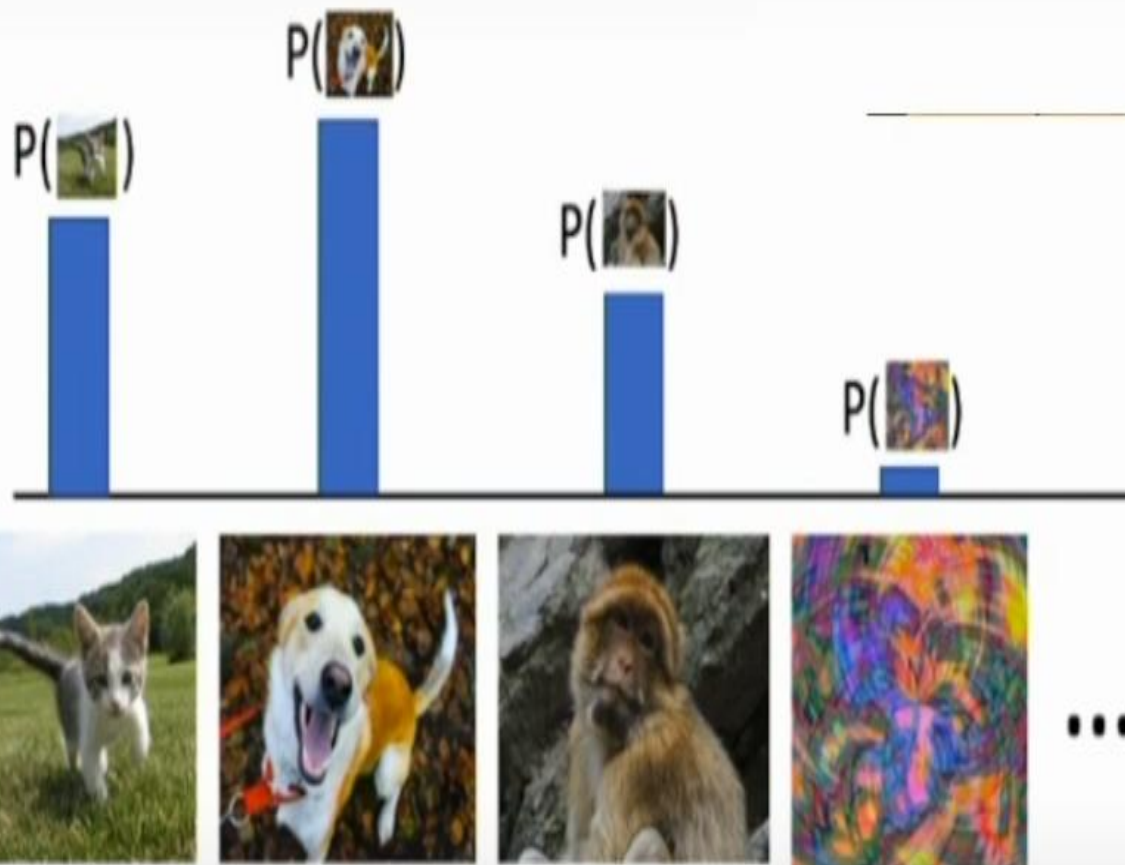
Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative

Model: Learn $p(x|y)$



Generative model: All possible images compete with each other for probability mass

Requires deep image understanding! Is a dog more likely to sit or stand? How about 3-legged dog vs 3-armed monkey?

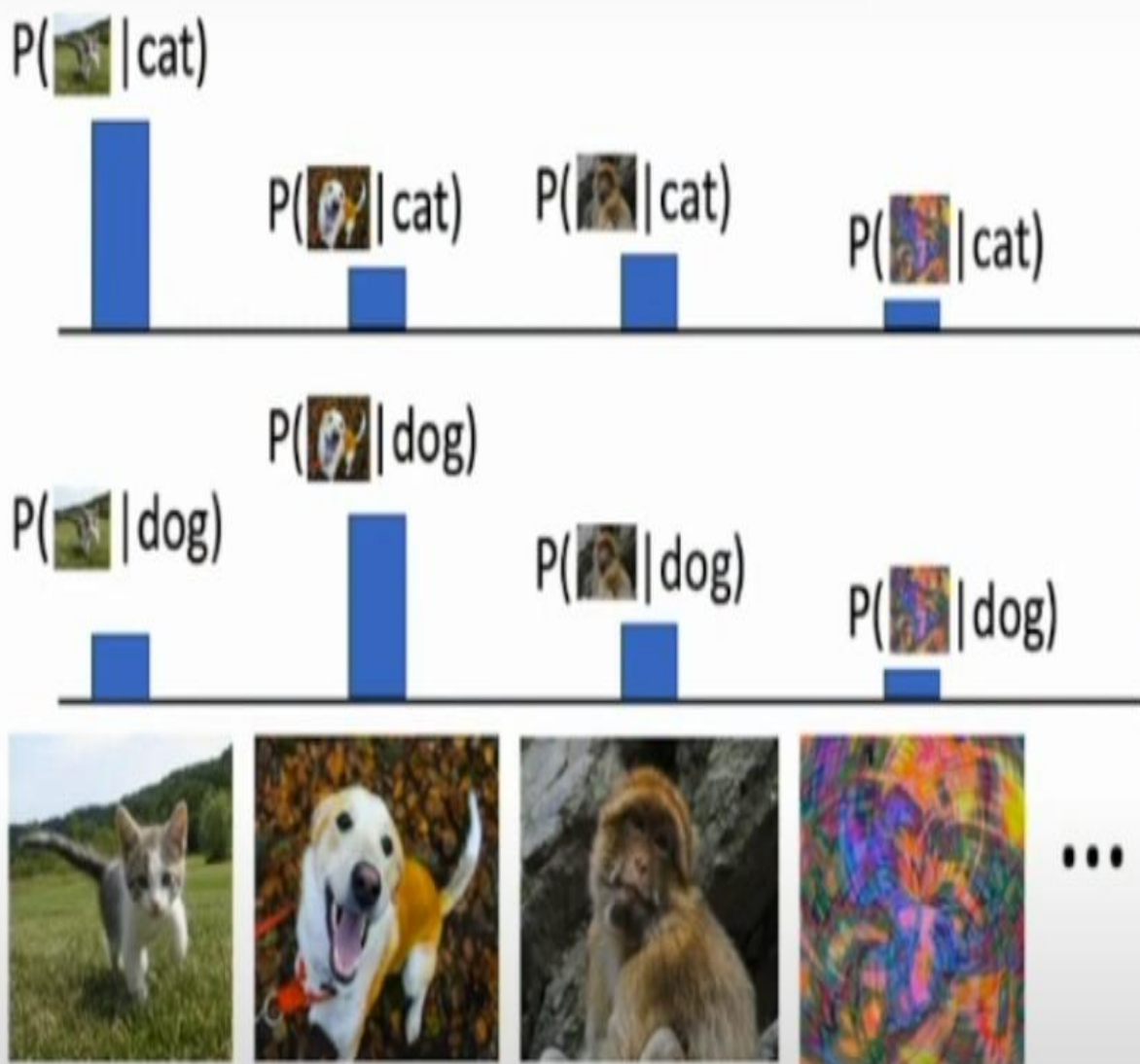
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$



Conditional Generative Model: Each possible label induces a competition among all images

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

Recall Bayes' Rule:

$$\underbrace{P(x | y)}_{\text{Conditional Generative Model}} = \frac{\overbrace{P(y | x)}^{\text{Discriminative Model}}}{\underbrace{P(y)}_{\text{Prior over labels}}} \underbrace{P(x)}_{\text{(Unconditional) Generative Model}}$$

We can build a conditional generative model from other components!

$P(x_k/C_i)$ is the number of tuples of class C_i in training set D having the value x_k , divided by number of tuples of class C_i in D

$X = (\text{Age} = 31..40$

$\text{Income} = \text{Low}$

$\text{student} = \text{No}$

$\text{Credit rating} =$

$\text{Excellent})$

Class:

$C_1: \text{buys_computer} = \text{'yes'}$

$C_2: \text{buys_computer} = \text{'no'}$

Data sample

$X = (\text{age} \leq 30,$

$\text{Income} = \text{medium},$

$\text{Student} = \text{yes}$

$\text{Credit_rating} = \text{Fair})$

age	income	student	credit_rating	com
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31...40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
> 40	medium	no	excellent	no

- $P(C_i)$: $P(\text{buys_computer} = \text{"yes"}) = 9/14 = 0.643$
 $P(\text{buys_computer} = \text{"no"}) = 5/14 = 0.357$
 - Compute $P(X|C_i)$ for each class
 - $P(\text{age} = \text{"<=30"} | \text{buys_computer} = \text{"yes"}) = 2/9 = 0.222$
 $P(\text{age} = \text{"<= 30"} | \text{buys_computer} = \text{"no"}) = 3/5 = 0.6$
 $P(\text{income} = \text{"medium"} | \text{buys_computer} = \text{"yes"}) = 4/9 = 0.444$
 $P(\text{income} = \text{"medium"} | \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$
 $P(\text{student} = \text{"yes"} | \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$
 $P(\text{student} = \text{"yes"} | \text{buys_computer} = \text{"no"}) = 1/5 = 0.2$
 $P(\text{credit_rating} = \text{"fair"} | \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$
 $P(\text{credit_rating} = \text{"fair"} | \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$
 - $X = (\text{age} \leq 30, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$
 - $P(X|C_i) : P(X|\text{buys_computer} = \text{"yes"}) = 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044$
 $P(X|\text{buys_computer} = \text{"no"}) = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.019$
 $P(X|C_i) * P(C_i) : P(X|\text{buys_computer} = \text{"yes"}) * P(\text{buys_computer} = \text{"yes"}) = 0.028$
 $P(X|\text{buys_computer} = \text{"no"}) * P(\text{buys_computer} = \text{"no"}) = 0.0036$
- Therefore, X belongs to class ("buys_computer = yes"**

$P(y)$

$P(X|y)$

$P(X|y)$

$$P(x|y) = \frac{P(y|x)}{P(y)} P(x)$$

Discriminative Model (Unconditional)
 Generative Model
 Conditional Generative Model
 Prior over labels

Discriminative Model:

Learn a probability distribution $p(y|x)$



Assign labels to data
Feature learning (supervised)

Generative Model:

Learn a probability distribution $p(x)$

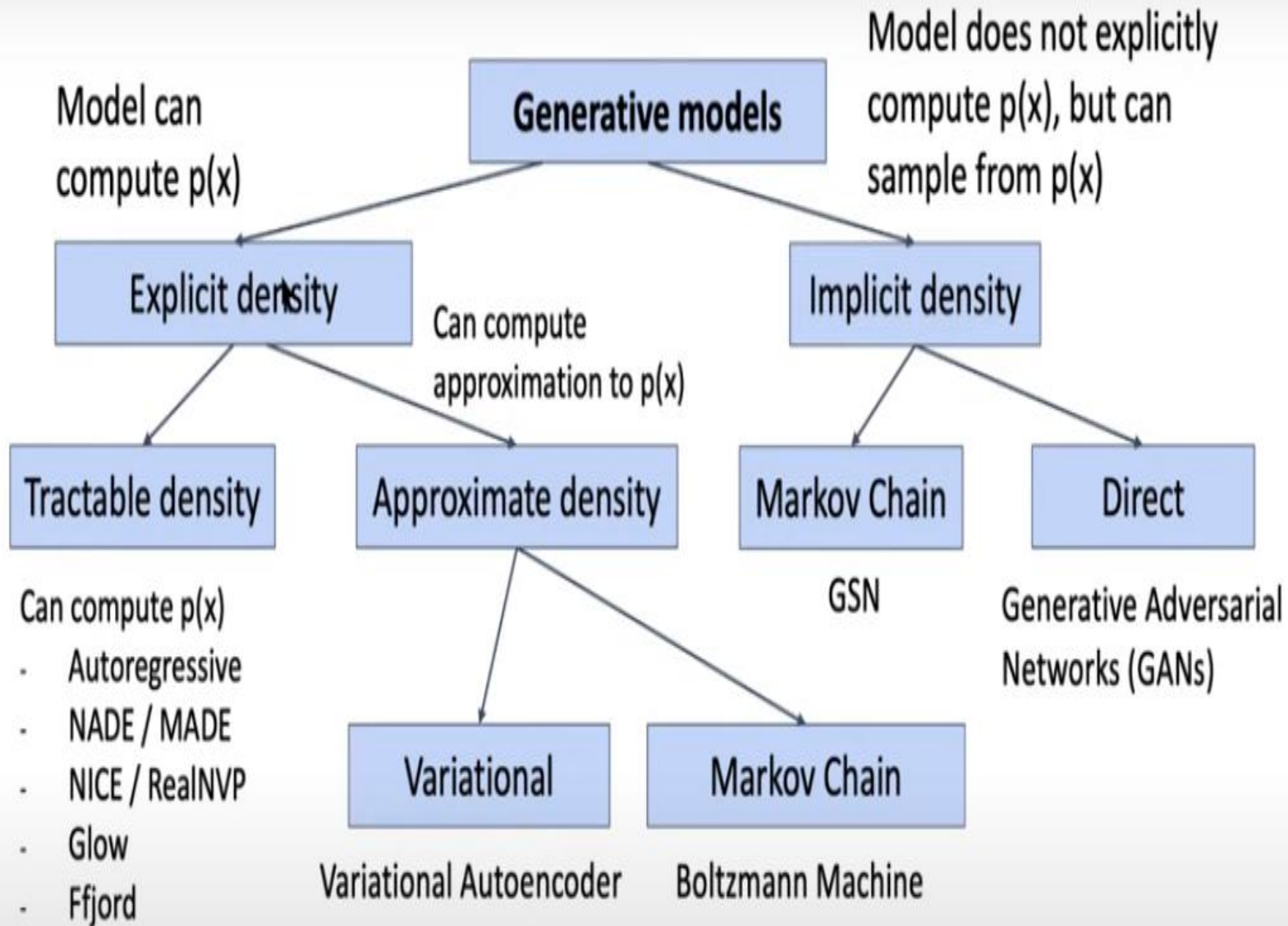


Detect outliers
Feature learning (unsupervised)
Sample to **generate** new data

Conditional Generative Model: Learn $p(x|y)$



Assign labels, while rejecting outliers!
Generate new data conditioned on input label



EXPLICIT

1. An **Explicit Likelihood Model** learns the data distribution from the sample distribution.
2. These types of models have access to a probability distribution, and they're trained using maximum likelihood or approximate likelihood models

IMPLICIT

1. **Implicit Likelihood Models** don't learn the distribution of the data, but rather learn the statistical properties of the data
2. It can generalise and generate new samples of data without depending on the probability distribution.

GANs ARE IMPLICIT GENERATIVE MODELS!

Explicit density : Allows likelihood inference

$$\boxed{P(x | y)} = \frac{\boxed{P(y | x)}}{\boxed{P(y)}} \boxed{P(x)}$$

Diagram illustrating the relationship between different probability models in a generative model framework:

- $P(x | y)$ is labeled as the **Conditional Generative Model** (blue box).
- $P(y | x)$ is labeled as the **Discriminative Model** (orange box).
- $P(y)$ is labeled as the **Prior over labels** (green box).
- $P(x)$ is labeled as the **(Unconditional) Generative Model** (purple box).

Implicit density: The latter targets a flexible transformation from random noise to generated samples. It uses transformer.

Generative Adversarial Network

1. Generative Adversarial Networks (GANs) are a powerful class of **neural networks** that are used for an **unsupervised_learning**.
2. GANs are made up of two neural_networks, a **discriminator and a generator**. They use adversarial training to produce artificial data that is identical to actual data.
3. The Generator attempts to fool the Discriminator, which is tasked with accurately distinguishing between produced and genuine data, by producing random noise samples.

Generative Adversarial Networks (GANs) can be broken down into three parts:

1. **Generative:** To learn a generative model, which describes how data is generated in terms of a probabilistic model.
2. **Adversarial:** The word adversarial refers to setting one thing up against another. This means that, in the context of GANs, the generative result is compared with the actual images in the data set. A mechanism known as a discriminator is used to apply a model that attempts to distinguish between real and fake images.
3. **Networks:** Use deep neural networks as artificial intelligence (AI) algorithms for training purposes.

Architecture of GANs

Generator Model

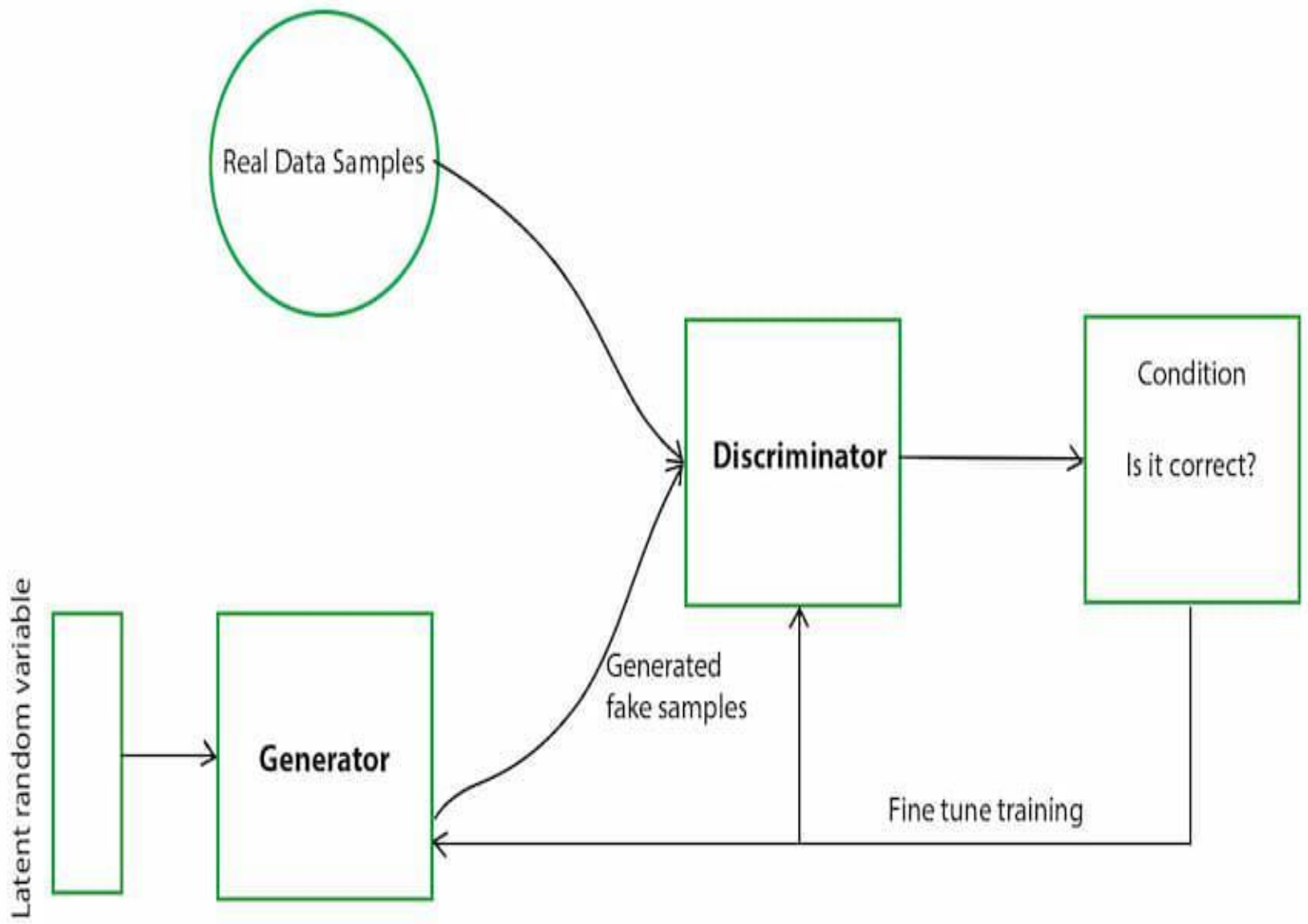
The generator takes random noise as input and converts it into complex data samples, such text or images. It is commonly depicted as a deep neural network.

Discriminator Model

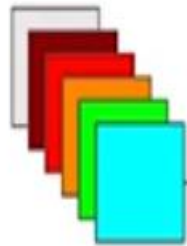
An artificial neural network called a discriminator model is used in Generative Adversarial Networks (GANs) to differentiate between generated and actual input. Convolutional layers are usually used in its architecture when dealing with picture data. Maximizing the discriminator's capacity to accurately identify generated samples as fraudulent and real samples as authentic is the aim of the adversarial training procedure.

Generator Loss

The loss is minimized when the log probability is maximized, when the discriminator is highly likely to classify the generated samples as real.



Samples



Latent Space



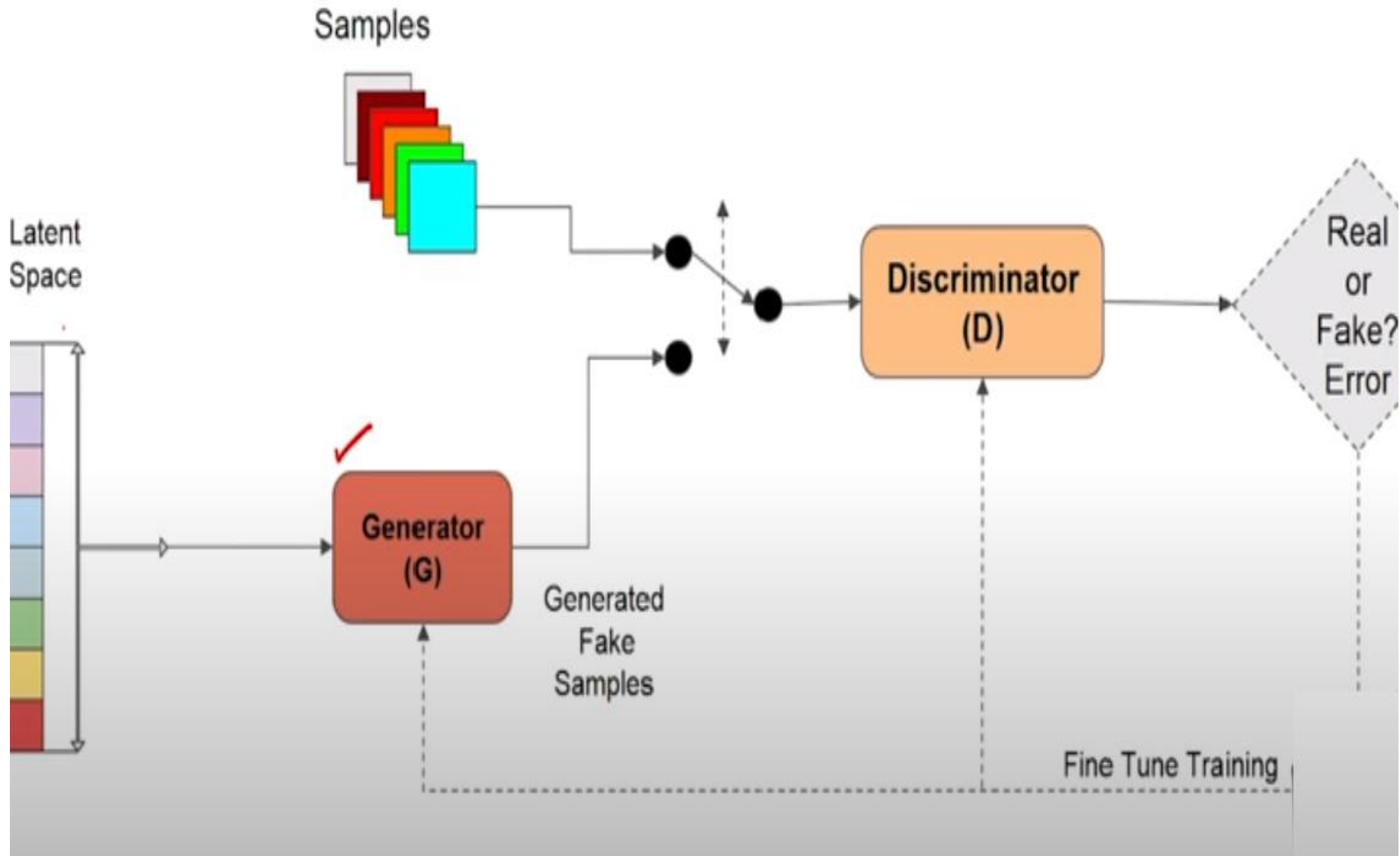
Generator
(G)

Generated
Fake
Samples

Discriminator
(D)

Real
or
Fake?
Error

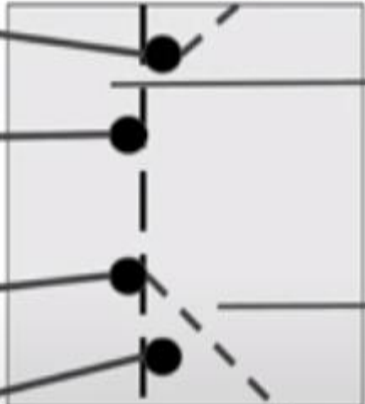
Fine Tune Training



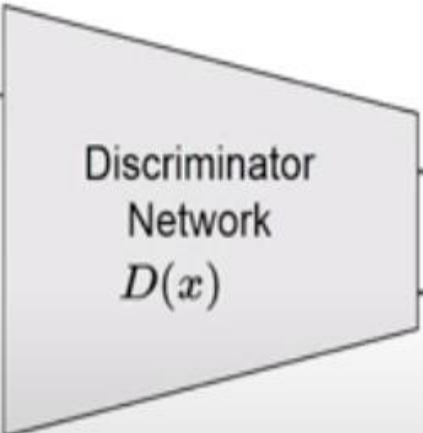
Real Data



Generated Data



$$D(x) \approx 0 \Rightarrow \log D(x) \ll 0$$



1
0

$$D(G(z)) \approx 1 \Rightarrow \log(1 - D(G(z))) \ll 0$$

MNIST
DATASET

5	0	4	1
2	1	3	1
3	5	3	6
7	2	8	6

Discriminator

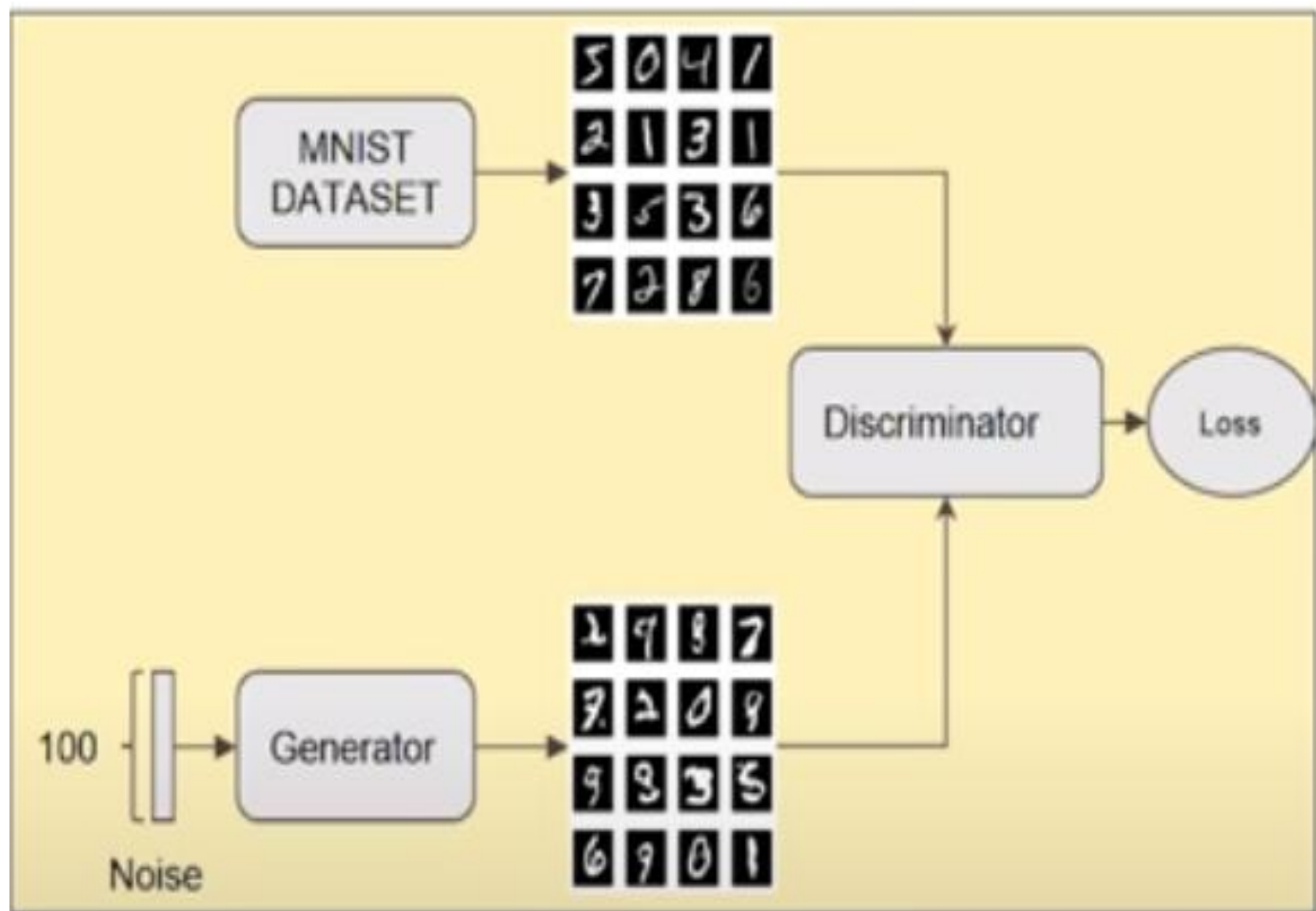
Loss

100

Noise

Generator

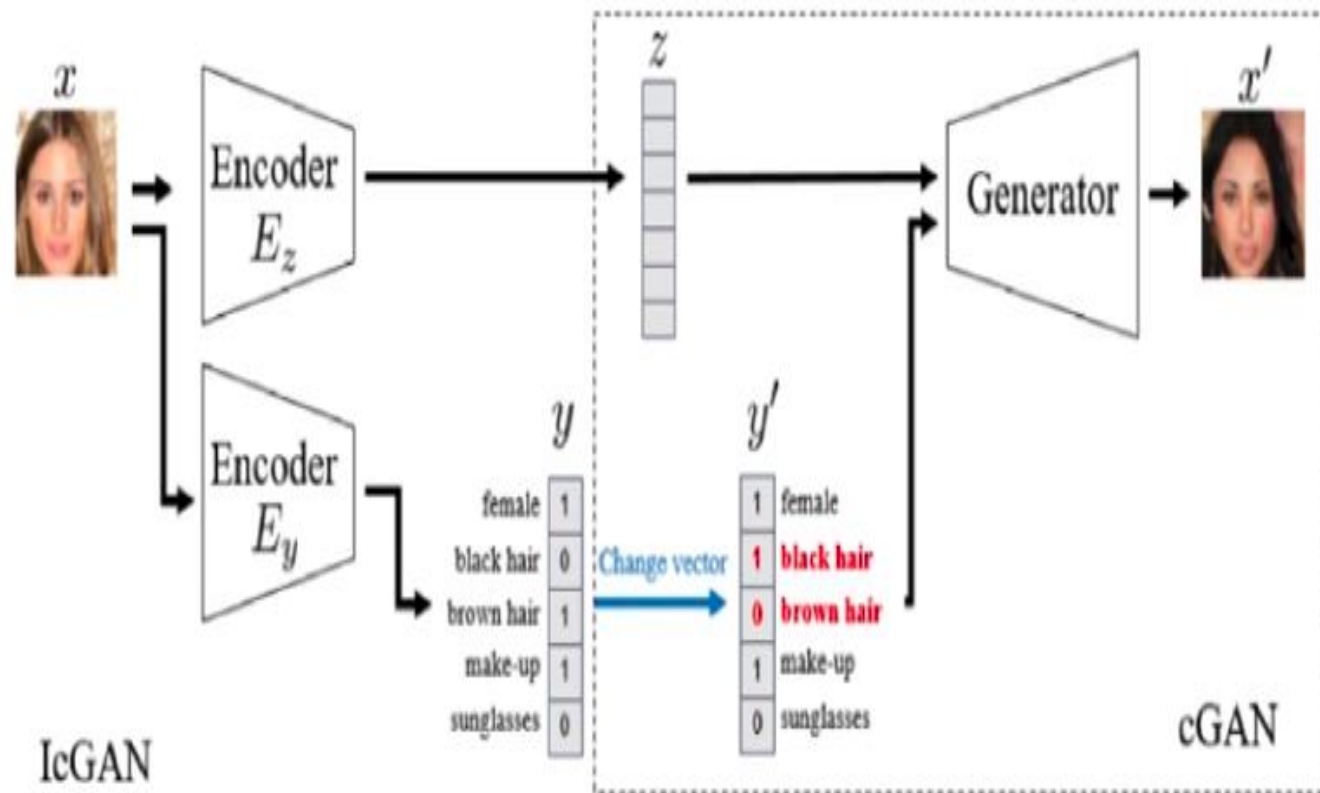
2	9	8	7
7	2	0	9
9	3	3	5
6	9	0	1



Types of GANs

- 1. Vanilla GAN:** This is the simplest type of GAN. Here, the Generator and the Discriminator are simple a basic multi-layer perceptrons. In vanilla GAN, the algorithm is really simple, it tries to optimize the mathematical equation using stochastic gradient descent.
- 2. Conditional GAN (CGAN):** CGAN can be described as a deep learning method in which **some conditional parameters are put into place**. In CGAN, an additional parameter 'y' is added to the Generator for generating the corresponding data.
- 3. Deep Convolutional GAN (DCGAN):**
DCGAN is one of the most popular and also the most successful implementations of GAN. It is composed of ConvNet in place of multi-layer perceptrons. The ConvNets are implemented without max pooling, which is in fact replaced by convolutional stride. Also, the layers are not fully connected.

- Combines a conditional GAN with an encoder which can encode an image to its latent representation



- Can be used for face editing as follows: (1) Pass image through encoder; (2) Change attribute vector y ; (3) Pass through generator

4. Laplacian Pyramid GAN (LAPGAN): The Laplacian_pyramid is a linear invertible image representation consisting of a set of band-pass images, spaced an octave apart, plus a low-frequency residual. This approach **uses multiple numbers of Generator and Discriminator networks** and different levels of the Laplacian Pyramid.

This approach is mainly used because it produces very high-quality images.

5. Super Resolution GAN (SRGAN): SRGAN as the name suggests is a way of designing a GAN in which a deep_neural_network is used along with an adversarial network in order to produce higher-resolution images.

Working of GAN:

Initialization: Two neural networks are created: a Generator (G) and a Discriminator (D).

G is tasked with creating new data, like images or text, that closely resembles real data.

D acts as a critic, trying to distinguish between real data (from a training dataset) and the data generated by G.

Generator's First Move: G takes a random noise vector as input. This noise vector contains random values and acts as the starting point for G's creation process. Using its internal layers and learned patterns, G transforms the noise vector into a new data sample, like a generated image.

Discriminator's Turn: D receives two kinds of inputs:

Real data samples from the training dataset.

The data samples generated by G in the previous step. D's job is to analyze each input and determine whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 indicates the data is likely real, and 0 suggests it's fake.

The Learning Process: Now, the adversarial part comes in:

If D correctly identifies real data as real (score close to 1) and generated data as fake (score close to 0), both G and D are rewarded to a small degree. This is because they're both doing their jobs well.

However, the key is to continuously improve. If D consistently identifies everything correctly, it won't learn much. So, the goal is for G to eventually trick D.

Generator's Improvement:

When D mistakenly labels G's creation as real (score close to 1), it's a sign that G is on the right track. In this case, G receives a significant positive update, while D receives a penalty for being fooled.

This feedback helps G improve its generation process to create more realistic data.

Discriminator's Adaptation:

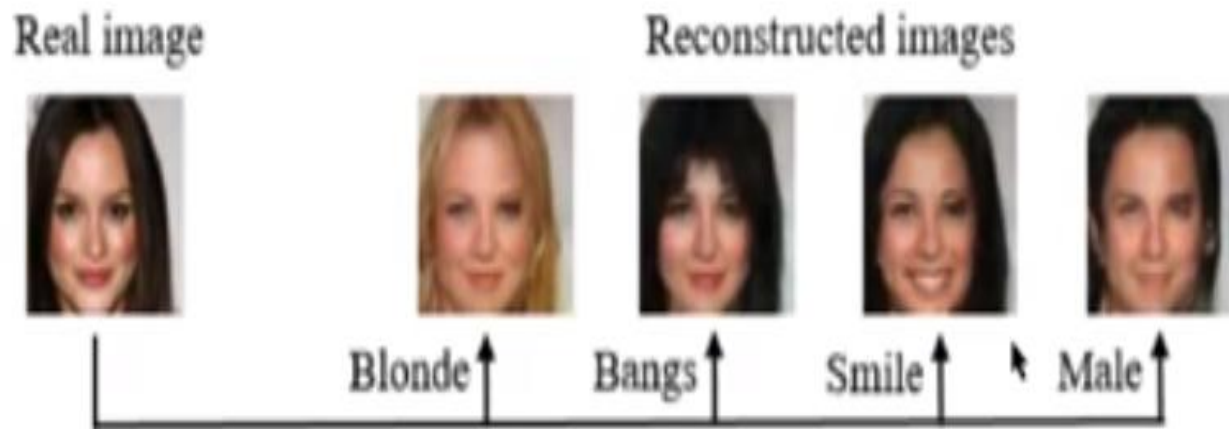
Conversely, if D correctly identifies G's fake data (score close to 0), but G receives no reward, D is further strengthened in its discrimination abilities.

This ongoing duel between G and D refines both networks over time.

Application Of Generative Adversarial Networks (GANs)

1. Image Synthesis and Generation

- Simple image edits such as grayscaling, brightness adjustment or contrast don't require an understanding of the image



- However, challenging operations such as changing attributes of a face require sound understanding of the input image. Can GANs help?
- But a GAN on its own does not have the mechanism to map a real image to its latent representation

Image-to-Image Translation

- **Image Super-Resolution:** Task of obtaining a High-Resolution (HR) image from a Low Resolution (LR) image, a challenging task
- **SRGAN (Super-Resolution GAN)** aims to generate photo-realistic images with 4× upscaling factors

4× SRGAN (proposed)



original



Advantages of GAN

The advantages of the GANs are as follows:

Synthetic data generation: GANs can generate new, synthetic data that resembles some known data distribution, which can be useful for data augmentation, anomaly detection, or creative applications.

High-quality results: GANs can produce high-quality, photorealistic results in image synthesis, video synthesis, music synthesis, and other tasks.

Unsupervised learning: GANs can be trained without labeled data, making them suitable for unsupervised learning tasks, where labeled data is scarce or difficult to obtain.

Versatility: GANs can be applied to a wide range of tasks, including image synthesis, text-to-image synthesis, image-to-image translation, anomaly detection, data augmentation, and others.

Disadvantages of GAN

The disadvantages of the GANs are as follows:

Training Instability: GANs can be difficult to train, with the risk of instability, mode collapse, or failure to converge.

Computational Cost: GANs can require a lot of computational resources and can be slow to train, especially for high-resolution images or large datasets.

Overfitting: GANs can overfit the training data, producing synthetic data that is too similar to the training data and lacking diversity.

Bias and Fairness: GANs can reflect the biases and unfairness present in the training data, leading to discriminatory or biased synthetic data.

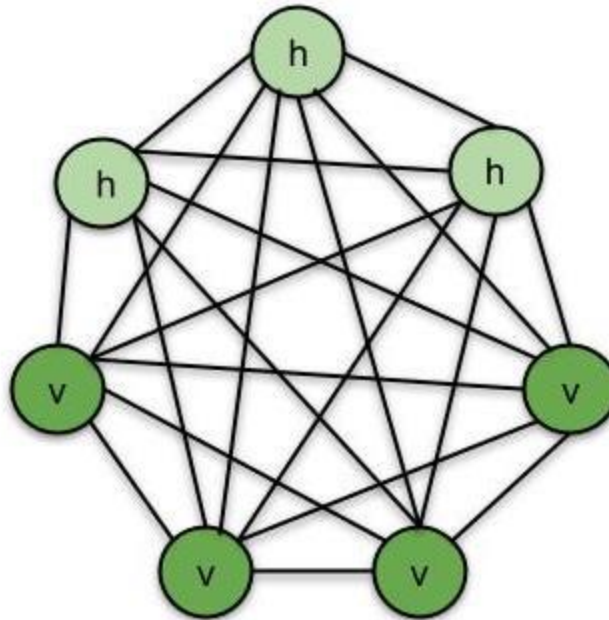
Interpretability and Accountability: GANs can be opaque and difficult to interpret or explain, making it challenging to ensure accountability, transparency, or fairness in their applications.

Boltzmann Machines is an unsupervised DL model in which every node is connected to every other node.

the Boltzmann Machines are **undirected** (or the connections are bidirectional). Boltzmann Machine is not a deterministic DL model but a **stochastic** or **generative** DL model

There are two types of nodes in the Boltzmann Machine — **Visible nodes** — those nodes which we can and do measure, and the **Hidden nodes** — those nodes which we cannot or do not measure.

The training data is fed into the Boltzmann Machine and the weights of the system are adjusted accordingly.

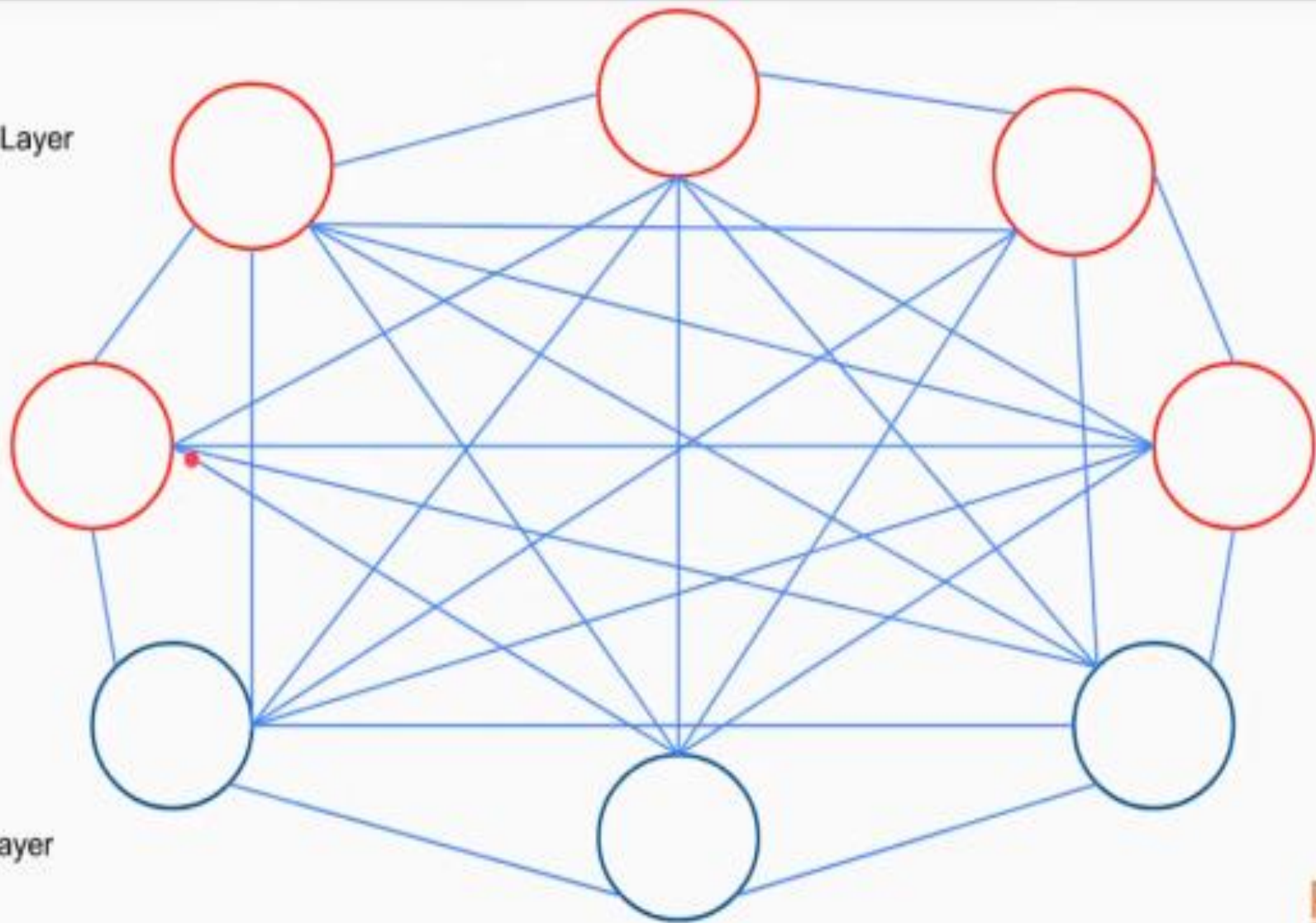


v - visible nodes, h - hidden nodes

- A Boltzmann machine is a network of symmetrically connected, neuron-like units that make stochastic decisions about whether to be on or off
- Undirected models, where connection goes both ways
- Non-deterministic model
- Purpose is to optimize the solution
- Discover features from datasets composed of binary vectors

Hidden Layer

Visible Layer



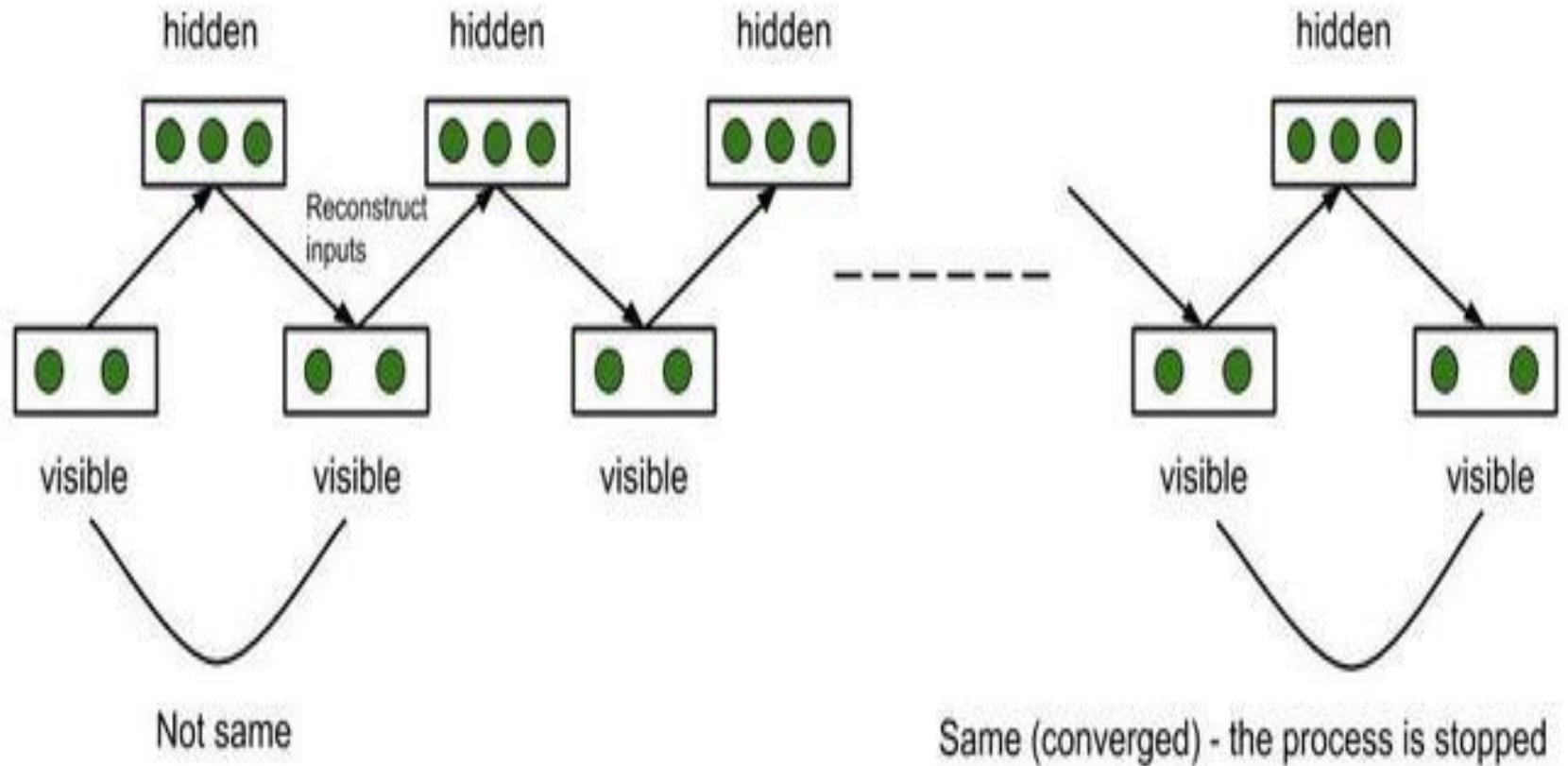
Types of Boltzmann Machines:

Restricted Boltzmann Machines (RBMs)

Deep Belief Networks (DBNs)

Deep Boltzmann Machines (DBMs)

Restricted Boltzmann Machines (RBMs)



They are designed to discover intricate structures within large datasets by learning to recreate the input data they're given. Think of a DBM as an artist who, after studying a collection of paintings, learns to create new artworks that could belong to the same collection. Similarly, a DBM analyzes data and learns how to produce new examples that are similar to the original data.

The 'deep' in the Deep Boltzmann Machine refers to the multiple layers in the network, which allow it to build a deep understanding of the data. Each layer captures increasingly abstract representations of the data. The first layer might detect edges in an image, the second layer might detect shapes, and the

